Google Data Analytics Capstone Project - Cyclistic

Sanketh Ks

February 2025

Introduction

This case study will use all the skills I developed in the Google Data Analytics Professional Certificate Course to complete the business tasks as a data analyst working for the fictional bike-sharing company **Cyclistic**. I will use the data analysis process of **Ask**, **Prepare**, **Process**, **Analyze**, **Share** and **Act** to answer key business questions so that the company can make data-driven decisions.

Scenario

The Director of Marketing, Lily Moreno, believes the Cyclistic's future success depends on maximizing the number of annual memberships. Therefore, the team wants to understand how casual riders and annual members use Cyclistic bikes differently. From these insights, the team will design a new marketing strategy to convert casual riders into annual members. But first, Cyclistic executives must approve the recommendations, so they must be backed up with compelling data insights and professional data visualizations.

About the Company

In 2016, Cyclistic launched a successful bike-share offering. Since then, the program has grown to a fleet of 5,824 bicycles that are geotracked and locked into a network of 692 stations across Chicago. The bikes can be unlocked from one station and returned to any other station in the system anytime.

Until now, Cyclistic's marketing strategy relied on building general awareness and appealing to broad consumer segments. One approach that helped make these things possible was the flexibility of its pricing plans: single-ride passes, full-day passes, and annual memberships. Customers who purchase single-ride or full-day passes are referred to as casual riders. Customers who purchase annual memberships are Cyclistic members.

Cyclistic's finance analysts have concluded that annual members are much more profitable than casual riders. Moreno believes that maximizing the number of annual members will be key to

future growth and she believes there is a very good chance to convert casual riders into members.

Moreno has set a clear goal: Design marketing strategies aimed at converting casual riders into annual members. In order to do that, however, the marketing analyst team needs to better understand the Cyclistic historical bike trip data to identify trends.

---

1. Ask

Business Task:

Analyze Cyclistic's August 2022 through July 2023 trip data to better understand how annual members and casual riders differ, then use insights to assist with marketing strategies aimed at converting casual riders into annual members.

Stakeholders

Lily Moreno: Director of Marketing. Moreno is responsible for the development of campaigns and initiatives to promote the bike-share program. These may include email, social media, and other channels.

Cyclistic marketing analytics team: A team of data analysts who are responsible for collecting, analyzing, and reporting data that helps guide Cyclistic marketing strategy.

Cyclistic executive team: The notoriously detail-oriented executive team will decide whether to approve the recommended marketing program.

---

2. Prepare

Trip data stored by Cyclistic goes back many years, but we will use data available for the last 12 months (August 2022 to July 2023). The data is broken up monthly and stored as 12 .zip files:

- 202208-divvy-tripdata.zip
- 202209-divvy-tripdata.zip
- 202210-divvy-tripdata.zip
- 202211-divvy-tripdata.zip
- 202212-divvy-tripdata.zip
- 202301-divvy-tripdata.zip
- 202302-divvy-tripdata.zip
- 202303-divvy-tripdata.zip
- 202304-divvy-tripdata.zip
- 202305-divvy-tripdata.zip
- 202306-divvy-tripdata.zip
- 202307-divvy-tripdata.zip

The data has been made available by Motivate International Inc. under this license.

The 12 .zip files were downloaded, stored, and extracted into the /Cyclistic/raw_trip_data/ directory.

---

3. Process

Load data analysis packages

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse
2.0.0 ──
## ✔ dplyr     1.1.2     ✔ readr     2.1.4
## ✔ forcats   1.0.0     ✔ stringr   1.5.0
## ✔ ggplot2   3.4.2     ✔ tibble    3.2.1
## ✔ lubridate 1.9.2     ✔ tidyr     1.3.0
## ✔ purrr     1.0.1
## ── Conflicts ────────────────────────────────────────── 
tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
library(janitor)
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##     chisq.test, fisher.test
```

```
library(scales)
```

```
##
## Attaching package: 'scales'
##
## The following object is masked from 'package:purrr':
##
##     discard
##
## The following object is masked from 'package:readr':
##
##     col_factor
```

```
library(leaflet)
```

```
## Warning: package 'leaflet' was built under R version 4.3.1
```

```
#library(hms)
```

Set working directory

```
setwd("D:/R/Cyclistic")
```

Import trip data from August 2022 to July 2023 into their respective dataframes

```
data_202208 <- read_csv("raw_trip_data/202208-divvy-tripdata.csv",
show_col_types = FALSE)
data_202209 <- read_csv("raw_trip_data/202209-divvy-publictripdata.csv",
show_col_types = FALSE)
data_202210 <- read_csv("raw_trip_data/202210-divvy-tripdata.csv",
show_col_types = FALSE)
data_202211 <- read_csv("raw_trip_data/202211-divvy-tripdata.csv",
show_col_types = FALSE)
data_202212 <- read_csv("raw_trip_data/202212-divvy-tripdata.csv",
show_col_types = FALSE)
data_202301 <- read_csv("raw_trip_data/202301-divvy-tripdata.csv",
show_col_types = FALSE)
data_202302 <- read_csv("raw_trip_data/202302-divvy-tripdata.csv",
show_col_types = FALSE)
data_202303 <- read_csv("raw_trip_data/202303-divvy-tripdata.csv",
show_col_types = FALSE)
data_202304 <- read_csv("raw_trip_data/202304-divvy-tripdata.csv",
show_col_types = FALSE)
data_202305 <- read_csv("raw_trip_data/202305-divvy-tripdata.csv",
show_col_types = FALSE)
data_202306 <- read_csv("raw_trip_data/202306-divvy-tripdata.csv",
show_col_types = FALSE)
data_202307 <- read_csv("raw_trip_data/202307-divvy-tripdata.csv",
show_col_types = FALSE)
```

Compare column names and data types to confirm they are the same in all 12 dataframes

```
compare_df_cols(data_202208, data_202209, data_202210, data_202211,
data_202212, data_202301, data_202302, data_202303, data_202304, data_202305,
data_202306, data_202307)
```

```
##           column_name     data_202208     data_202209     data_202210
## 1             end_lat         numeric         numeric         numeric
## 2             end_lng         numeric         numeric         numeric
## 3      end_station_id       character       character       character
## 4    end_station_name       character       character       character
## 5            ended_at POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt
## 6       member_casual       character       character       character
## 7             ride_id       character       character       character
## 8       rideable_type       character       character       character
## 9           start_lat         numeric         numeric         numeric
```

```
## 10          start_lng         numeric         numeric         numeric
## 11   start_station_id       character       character       character
## 12 start_station_name       character       character       character
## 13         started_at POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt
##        data_202211     data_202212     data_202301     data_202302
## 1          numeric         numeric         numeric         numeric
## 2          numeric         numeric         numeric         numeric
## 3        character       character       character       character
## 4        character       character       character       character
## 5  POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt
## 6        character       character       character       character
## 7        character       character       character       character
## 8        character       character       character       character
## 9          numeric         numeric         numeric         numeric
## 10         numeric         numeric         numeric         numeric
## 11       character       character       character       character
## 12       character       character       character       character
## 13 POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt
##        data_202303     data_202304     data_202305     data_202306
## 1          numeric         numeric         numeric         numeric
## 2          numeric         numeric         numeric         numeric
## 3        character       character       character       character
## 4        character       character       character       character
## 5  POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt
## 6        character       character       character       character
## 7        character       character       character       character
## 8        character       character       character       character
## 9          numeric         numeric         numeric         numeric
## 10         numeric         numeric         numeric         numeric
## 11       character       character       character       character
## 12       character       character       character       character
## 13 POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt POSIXct, POSIXt
##        data_202307
## 1          numeric
## 2          numeric
## 3        character
## 4        character
## 5  POSIXct, POSIXt
## 6        character
## 7        character
## 8        character
## 9          numeric
## 10         numeric
## 11       character
## 12       character
## 13 POSIXct, POSIXt
```

Column names and data types match so the 12 dataframes will be merged into 1

```
data_trip <- rbind(data_202208, data_202209, data_202210, data_202211,
data_202212, data_202301, data_202302, data_202303, data_202304, data_202305,
data_202306, data_202307)
#data_trip <- data_202208
```

Remove the 12 monthly dataframes to free up memory

```
rm(data_202208, data_202209, data_202210, data_202211, data_202212,
data_202301, data_202302, data_202303, data_202304, data_202305, data_202306,
data_202307)
```

Check merged data

```
head(data_trip)
```

```
## # A tibble: 6 × 13
##   ride_id          rideable_type started_at          ended_at
##   <chr>            <chr>         <dttm>              <dttm>
## 1 550CF7EFEAE0C618 electric_bike 2022-08-07 21:34:15 2022-08-07 21:41:46
## 2 DAD198F405F9C5F5 electric_bike 2022-08-08 14:39:21 2022-08-08 14:53:23
## 3 E6F2BC47B65CB7FD electric_bike 2022-08-08 15:29:50 2022-08-08 15:40:34
## 4 F597830181C2E13C electric_bike 2022-08-08 02:43:50 2022-08-08 02:58:53
## 5 0CE689BB4E313E8D electric_bike 2022-08-07 20:24:06 2022-08-07 20:29:58
## 6 BFA7E7CC69860C20 electric_bike 2022-08-08 13:06:08 2022-08-08 13:19:09
## # i 9 more variables: start_station_name <chr>, start_station_id <chr>,
## #   end_station_name <chr>, end_station_id <chr>, start_lat <dbl>,
## #   start_lng <dbl>, end_lat <dbl>, end_lng <dbl>, member_casual <chr>
```

```
str(data_trip)
```

```
## spc_tbl_ [5,723,606 × 13] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ ride_id           : chr [1:5723606] "550CF7EFEAE0C618"
"DAD198F405F9C5F5" "E6F2BC47B65CB7FD" "F597830181C2E13C" ...
##  $ rideable_type     : chr [1:5723606] "electric_bike" "electric_bike"
"electric_bike" "electric_bike" ...
##  $ started_at        : POSIXct[1:5723606], format: "2022-08-07 21:34:15"
"2022-08-08 14:39:21" ...
##  $ ended_at          : POSIXct[1:5723606], format: "2022-08-07 21:41:46"
"2022-08-08 14:53:23" ...
##  $ start_station_name: chr [1:5723606] NA NA NA NA ...
##  $ start_station_id  : chr [1:5723606] NA NA NA NA ...
##  $ end_station_name  : chr [1:5723606] NA NA NA NA ...
##  $ end_station_id    : chr [1:5723606] NA NA NA NA ...
##  $ start_lat         : num [1:5723606] 41.9 41.9 42 41.9 41.9 ...
##  $ start_lng         : num [1:5723606] -87.7 -87.6 -87.7 -87.7 -87.7 ...
##  $ end_lat           : num [1:5723606] 41.9 41.9 42 42 41.8 ...
##  $ end_lng           : num [1:5723606] -87.7 -87.6 -87.7 -87.7 -87.7 ...
##  $ member_casual     : chr [1:5723606] "casual" "casual" "casual" "casual"
...
##  - attr(*, "spec")=
##   .. cols(
##   ..   ride_id = col_character(),
```

```
##   ..   rideable_type = col_character(),
##   ..   started_at = col_datetime(format = ""),
##   ..   ended_at = col_datetime(format = ""),
##   ..   start_station_name = col_character(),
##   ..   start_station_id = col_character(),
##   ..   end_station_name = col_character(),
##   ..   end_station_id = col_character(),
##   ..   start_lat = col_double(),
##   ..   start_lng = col_double(),
##   ..   end_lat = col_double(),
##   ..   end_lng = col_double(),
##   ..   member_casual = col_character()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

Remove start_station_name, start_station_id, end_station_name, end_station_id because values are not consistently filled in. We can use start/end latitude and start/end longitude to determine locations

```
data_trip <- data_trip %>%
  select(-c(start_station_name, start_station_id, end_station_name,
end_station_id))
```

Number of missing values in dataframe

```
sum(is.na(data_trip))
```

```
## [1] 12204
```

Remove the rows of data that have missing values

```
data_trip <- na.omit(data_trip)
```

Check for duplicate rows

```
#remove comment tag later
#duplicate_rows <- data_trip[duplicated(data_trip), ]
#str(duplicate_rows)
#rm(duplicate_rows)
```

Create a new column named ride_length (in minutes) by taking the difference between ended_at and started_at

```
data_trip$ride_length <-
as.numeric(as.character(round(difftime(data_trip$ended_at,
data_trip$started_at, units="mins"), 2)))
#data_trip$ride_length <- as_hms(difftime(data_trip$ended_at,
data_trip$started_at))
str(data_trip)
```

```
## tibble [5,717,504 × 10] (S3: tbl_df/tbl/data.frame)
##  $ ride_id      : chr [1:5717504] "550CF7EFEAE0C618" "DAD198F405F9C5F5"
"E6F2BC47B65CB7FD" "F597830181C2E13C" ...
##  $ rideable_type: chr [1:5717504] "electric_bike" "electric_bike"
"electric_bike" "electric_bike" ...
##  $ started_at   : POSIXct[1:5717504], format: "2022-08-07 21:34:15"
"2022-08-08 14:39:21" ...
##  $ ended_at     : POSIXct[1:5717504], format: "2022-08-07 21:41:46"
"2022-08-08 14:53:23" ...
##  $ start_lat    : num [1:5717504] 41.9 41.9 42 41.9 41.9 ...
##  $ start_lng    : num [1:5717504] -87.7 -87.6 -87.7 -87.7 -87.7 ...
##  $ end_lat      : num [1:5717504] 41.9 41.9 42 42 41.8 ...
##  $ end_lng      : num [1:5717504] -87.7 -87.6 -87.7 -87.7 -87.7 ...
##  $ member_casual: chr [1:5717504] "casual" "casual" "casual" "casual" ...
##  $ ride_length  : num [1:5717504] 7.52 14.03 10.73 15.05 5.87 ...
##  - attr(*, "na.action")= 'omit' Named int [1:6102] 8293 17483 84582 84608
84788 84820 85004 85043 85351 85564 ...
##   ..- attr(*, "names")= chr [1:6102] "8293" "17483" "84582" "84608" ...
```

Check for zero or negative ride_length values

```
negative_rows <- data_trip[data_trip$ride_length<=0, ]
str(negative_rows)
```

```
## tibble [715 × 10] (S3: tbl_df/tbl/data.frame)
##  $ ride_id      : chr [1:715] "162024EA2A5F23AC" "8B5EDA7571220F96"
"09B71177D92C0DC5" "968C67D0CE946110" ...
##  $ rideable_type: chr [1:715] "electric_bike" "electric_bike"
"classic_bike" "electric_bike" ...
##  $ started_at   : POSIXct[1:715], format: "2022-08-18 18:16:28"
"2022-08-05 18:28:39" ...
##  $ ended_at     : POSIXct[1:715], format: "2022-08-18 18:16:28"
"2022-08-05 18:28:39" ...
##  $ start_lat    : num [1:715] 41.8 42 41.9 42 41.9 ...
##  $ start_lng    : num [1:715] -87.6 -87.7 -87.6 -87.7 -87.6 ...
##  $ end_lat      : num [1:715] 41.8 42 41.9 42 41.9 ...
##  $ end_lng      : num [1:715] -87.6 -87.7 -87.6 -87.7 -87.6 ...
##  $ member_casual: chr [1:715] "member" "member" "casual" "casual" ...
##  $ ride_length  : num [1:715] 0 0 0 0 0 0 0 0 0 0 ...
##  - attr(*, "na.action")= 'omit' Named int [1:6102] 8293 17483 84582 84608
84788 84820 85004 85043 85351 85564 ...
##   ..- attr(*, "names")= chr [1:6102] "8293" "17483" "84582" "84608" ...
```

```
rm(negative_rows)
```

Remove rows with zero or negative value ride_length

```
data_trip <- data_trip[!(data_trip$ride_length<=0), ]
str(data_trip)
```

```
## tibble [5,716,789 × 10] (S3: tbl_df/tbl/data.frame)
##  $ ride_id      : chr [1:5716789] "550CF7EFEAE0C618" "DAD198F405F9C5F5"
"E6F2BC47B65CB7FD" "F597830181C2E13C" ...
##  $ rideable_type: chr [1:5716789] "electric_bike" "electric_bike"
"electric_bike" "electric_bike" ...
##  $ started_at   : POSIXct[1:5716789], format: "2022-08-07 21:34:15"
"2022-08-08 14:39:21" ...
##  $ ended_at     : POSIXct[1:5716789], format: "2022-08-07 21:41:46"
"2022-08-08 14:53:23" ...
##  $ start_lat    : num [1:5716789] 41.9 41.9 42 41.9 41.9 ...
##  $ start_lng    : num [1:5716789] -87.7 -87.6 -87.7 -87.7 -87.7 ...
##  $ end_lat      : num [1:5716789] 41.9 41.9 42 42 41.8 ...
##  $ end_lng      : num [1:5716789] -87.7 -87.6 -87.7 -87.7 -87.7 ...
##  $ member_casual: chr [1:5716789] "casual" "casual" "casual" "casual" ...
##  $ ride_length  : num [1:5716789] 7.52 14.03 10.73 15.05 5.87 ...
##  - attr(*, "na.action")= 'omit' Named int [1:6102] 8293 17483 84582 84608
84788 84820 85004 85043 85351 85564 ...
##   ..- attr(*, "names")= chr [1:6102] "8293" "17483" "84582" "84608" ...
```

Summary to display minimum, median, mean, maximum, 1st quartile, and 3rd quartile for ride_length in minutes

```
summary(time_length(data_trip$ride_length), unit = "mins" )
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
##     0.02     5.45     9.58    15.17    17.05 12136.30
```

Sort ride_length by descending order to find incongruent values

```
newdata <- data_trip[order(-data_trip$ride_length), ]
head(newdata$ride_length, n = 20)
```

```
##  [1] 12136.30 11152.27  8243.80  4903.12  4848.35  3818.40  3245.78
2719.93
##  [9]  2457.85  2349.35  1767.38  1499.93  1499.93  1499.93  1499.92
1499.92
## [17]  1499.92  1499.92  1499.92  1499.90
```

It is possible the rider did not return the bike because they needed it for that length of time so we will leave those values in place.

```
remove(newdata)
```

Create new columns named date (YYY-MM-DD), year (YYYY), month(MM), day(DD), day_of_week (Sunday, Monday, etc.) and hour_started (by hour) using started_at; create coords_start and coords_end

```
data_trip$date <- as.Date(data_trip$started_at)
data_trip$year <- format(as.Date(data_trip$date), "%Y")
data_trip$month <- format(as.Date(data_trip$date), "%m")
data_trip$day <- format(as.Date(data_trip$date), "%d")
data_trip$day_of_week <- format(as.Date(data_trip$date), "%A")
data_trip$hour_started <- hour(data_trip$started_at)
data_trip$coords_start <- paste(data_trip$start_lat, data_trip$start_lng,
sep=", ")
data_trip$coords_end <- paste(data_trip$end_lat, data_trip$end_lng, sep=", ")
str(data_trip)
```

```
## tibble [5,716,789 × 18] (S3: tbl_df/tbl/data.frame)
##  $ ride_id      : chr [1:5716789] "550CF7EFEAE0C618" "DAD198F405F9C5F5"
"E6F2BC47B65CB7FD" "F597830181C2E13C" ...
##  $ rideable_type: chr [1:5716789] "electric_bike" "electric_bike"
"electric_bike" "electric_bike" ...
##  $ started_at   : POSIXct[1:5716789], format: "2022-08-07 21:34:15"
"2022-08-08 14:39:21" ...
##  $ ended_at     : POSIXct[1:5716789], format: "2022-08-07 21:41:46"
"2022-08-08 14:53:23" ...
##  $ start_lat    : num [1:5716789] 41.9 41.9 42 41.9 41.9 ...
##  $ start_lng    : num [1:5716789] -87.7 -87.6 -87.7 -87.7 -87.7 ...
##  $ end_lat      : num [1:5716789] 41.9 41.9 42 42 41.8 ...
##  $ end_lng      : num [1:5716789] -87.7 -87.6 -87.7 -87.7 -87.7 ...
##  $ member_casual: chr [1:5716789] "casual" "casual" "casual" "casual" ...
##  $ ride_length  : num [1:5716789] 7.52 14.03 10.73 15.05 5.87 ...
##  $ date         : Date[1:5716789], format: "2022-08-07" "2022-08-08" ...
##  $ year         : chr [1:5716789] "2022" "2022" "2022" "2022" ...
##  $ month        : chr [1:5716789] "08" "08" "08" "08" ...
##  $ day          : chr [1:5716789] "07" "08" "08" "08" ...
##  $ day_of_week  : chr [1:5716789] "Sunday" "Monday" "Monday" "Monday" ...
##  $ hour_started : int [1:5716789] 21 14 15 2 20 13 14 20 21 23 ...
##  $ coords_start : chr [1:5716789] "41.93, -87.69" "41.89, -87.64" "41.97,
-87.69" "41.94, -87.65" ...
##  $ coords_end   : chr [1:5716789] "41.94, -87.72" "41.92, -87.64" "41.97,
-87.66" "41.97, -87.69" ...
##  - attr(*, "na.action")= 'omit' Named int [1:6102] 8293 17483 84582 84608
84788 84820 85004 85043 85351 85564 ...
##   ..- attr(*, "names")= chr [1:6102] "8293" "17483" "84582" "84608" ...
```

---

4. Analyze

- Top Start and End Stations per rider type

Total trips by rider type

```
data_trip %>%
  group_by(member_casual) %>%
  summarize(total_count = n())
```

```
## # A tibble: 2 × 2
##   member_casual total_count
##   <chr>               <int>
## 1 casual            2163955
## 2 member            3552834
```

Total trips per bike type

```
data_trip %>%
  group_by(rideable_type) %>%
  summarize(total_count = n())
```

```
## # A tibble: 3 × 2
##   rideable_type total_count
##   <chr>               <int>
## 1 classic_bike      2480891
## 2 docked_bike        126089
## 3 electric_bike     3109809
```

Total trips per bike type for each type of rider

```
data_trip %>%
  group_by(member_casual, rideable_type) %>%
  summarize(total_count = n())
```

```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```

```
## # A tibble: 5 × 3
## # Groups:   member_casual [2]
##   member_casual rideable_type total_count
##   <chr>         <chr>               <int>
## 1 casual        classic_bike       788656
## 2 casual        docked_bike        126089
## 3 casual        electric_bike     1249210
## 4 member        classic_bike      1692235
## 5 member        electric_bike     1860599
```

Total trips by year and month

```
data_trip %>%
  group_by(month, year) %>%
  summarize(total_count = n()) %>%
  arrange(year)
```

```
## `summarise()` has grouped output by 'month'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 12 × 3
## # Groups:   month [12]
##    month year  total_count
##    <chr> <chr>       <int>
##  1 08    2022       785012
##  2 09    2022       700555
##  3 10    2022       558145
##  4 11    2022       337447
##  5 12    2022       181664
##  6 01    2023       190166
##  7 02    2023       190320
##  8 03    2023       258476
##  9 04    2023       426119
## 10 05    2023       604046
## 11 06    2023       718656
## 12 07    2023       766183
```

Order day of the week to start on Sunday and end on Saturday

```
data_trip$day_of_week <- factor(data_trip$day_of_week,
  levels = c("Sunday", "Monday", "Tuesday", "Wednesday", "Thursday",
"Friday", "Saturday"))
```

Total trips by day of the week

```
data_trip %>%
  group_by(day_of_week) %>%
  summarize(total_count = n())
```

```
## # A tibble: 7 × 2
##   day_of_week total_count
##   <fct>             <int>
## 1 Sunday           717947
## 2 Monday           758647
## 3 Tuesday          808072
## 4 Wednesday        827579
## 5 Thursday         858326
## 6 Friday           850829
## 7 Saturday         895389
```

Total trips by day of the week for each rider type

```
data_trip$day_of_week <- factor(data_trip$day_of_week,
  levels = c("Sunday", "Monday", "Tuesday", "Wednesday", "Thursday",
"Friday", "Saturday"))
data_trip %>%
```

```
  group_by(member_casual, day_of_week) %>%
  summarize(total_count = n())
```

```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```

```
## # A tibble: 14 × 3
## # Groups:   member_casual [2]
##    member_casual day_of_week total_count
##    <chr>         <fct>             <int>
##  1 casual        Sunday           331577
##  2 casual        Monday           257332
##  3 casual        Tuesday          256287
##  4 casual        Wednesday        261827
##  5 casual        Thursday         288609
##  6 casual        Friday           334276
##  7 casual        Saturday         434047
##  8 member        Sunday           386370
##  9 member        Monday           501315
## 10 member        Tuesday          551785
## 11 member        Wednesday        565752
## 12 member        Thursday         569717
## 13 member        Friday           516553
## 14 member        Saturday         461342
```

Total trips by hour for each rider type

```
data_trip %>%
  group_by(member_casual, hour_started) %>%
  summarize(total_count = n()) %>%
  as_tibble() %>%
  print(n=48)
```

```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```

```
## # A tibble: 48 × 3
##    member_casual hour_started total_count
##    <chr>                <int>       <int>
##  1 casual                   0       40834
##  2 casual                   1       26474
##  3 casual                   2       15992
##  4 casual                   3        9082
##  5 casual                   4        6411
##  6 casual                   5       12070
##  7 casual                   6       31840
##  8 casual                   7       53988
##  9 casual                   8       72378
## 10 casual                   9       71657
## 11 casual                  10       88846
## 12 casual                  11      114264
```

```
## 13 casual                  12      135132
## 14 casual                  13      140860
## 15 casual                  14      149734
## 16 casual                  15      167895
## 17 casual                  16      189782
## 18 casual                  17      209310
## 19 casual                  18      182695
## 20 casual                  19      136144
## 21 casual                  20       97979
## 22 casual                  21       83217
## 23 casual                  22       73106
## 24 casual                  23       54265
## 25 member                   0       36122
## 26 member                   1       21990
## 27 member                   2       12667
## 28 member                   3        7989
## 29 member                   4        8751
## 30 member                   5       32852
## 31 member                   6      101195
## 32 member                   7      185982
## 33 member                   8      231075
## 34 member                   9      157495
## 35 member                  10      144358
## 36 member                  11      170785
## 37 member                  12      194904
## 38 member                  13      193966
## 39 member                  14      195483
## 40 member                  15      238754
## 41 member                  16      317839
## 42 member                  17      375461
## 43 member                  18      299963
## 44 member                  19      215343
## 45 member                  20      149373
## 46 member                  21      116502
## 47 member                  22       87312
## 48 member                  23       56673
```

Average ride length for each rider type (in minutes)

```
data_trip %>%
  group_by(member_casual) %>%
  summarize(avg_ride_length = mean(ride_length))
```

```
## # A tibble: 2 × 2
##   member_casual avg_ride_length
##   <chr>                   <dbl>
## 1 casual                   20.3
## 2 member                   12.0
```

Average ride length for each bike type (in minutes)

```
data_trip %>%
```

```
  group_by(rideable_type) %>%
  summarize(avg_ride_length = mean(ride_length))
```

```
## # A tibble: 3 × 2
##   rideable_type avg_ride_length
##   <chr>                   <dbl>
## 1 classic_bike             16.7
## 2 docked_bike              49.4
## 3 electric_bike            12.5
```

Average ride length for each rider type by bike type (in minutes)

```
data_trip %>%
  group_by(member_casual, rideable_type) %>%
  summarize(avg_ride_length = mean(ride_length))
```

```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```

```
## # A tibble: 5 × 3
## # Groups:   member_casual [2]
##   member_casual rideable_type avg_ride_length
##   <chr>         <chr>                   <dbl>
## 1 casual        classic_bike             24.6
## 2 casual        docked_bike              49.4
## 3 casual        electric_bike            14.7
## 4 member        classic_bike             13.0
## 5 member        electric_bike            11.1
```

Average ride length for each rider type by day of the week (in minutes)

```
data_trip %>%
  group_by(member_casual, day_of_week) %>%
  summarize(avg_ride_length = mean(ride_length))
```

```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```

```
## # A tibble: 14 × 3
## # Groups:   member_casual [2]
##    member_casual day_of_week avg_ride_length
##    <chr>         <fct>                 <dbl>
##  1 casual        Sunday                 23.2
##  2 casual        Monday                 20.1
##  3 casual        Tuesday                18.4
##  4 casual        Wednesday              17.4
##  5 casual        Thursday               18.0
```

```
##  6 casual        Friday                 19.8
##  7 casual        Saturday               23.0
##  8 member        Sunday                 13.3
##  9 member        Monday                 11.5
## 10 member        Tuesday                11.6
## 11 member        Wednesday              11.5
## 12 member        Thursday               11.6
## 13 member        Friday                 12.0
## 14 member        Saturday               13.4
```

Average ride length for each rider type by year and month (in minutes)

```
data_trip %>%
  group_by(member_casual, year, month) %>%
  summarize(avg_ride_length = mean(ride_length)) %>%
  as_tibble() %>%
  print(n=24)
```

```
## `summarise()` has grouped output by 'member_casual', 'year'. You can
override
## using the `.groups` argument.
```

```
## # A tibble: 24 × 4
##    member_casual year  month avg_ride_length
##    <chr>         <chr> <chr>           <dbl>
##  1 casual        2022  08               21.4
##  2 casual        2022  09               20.0
##  3 casual        2022  10               18.4
##  4 casual        2022  11               15.5
##  5 casual        2022  12               13.4
##  6 casual        2023  01               13.6
##  7 casual        2023  02               15.9
##  8 casual        2023  03               15.2
##  9 casual        2023  04               20.4
## 10 casual        2023  05               22.0
## 11 casual        2023  06               21.7
## 12 casual        2023  07               22.7
## 13 member        2022  08               13.1
## 14 member        2022  09               12.6
## 15 member        2022  10               11.5
## 16 member        2022  11               10.9
## 17 member        2022  12               10.4
## 18 member        2023  01               10.1
## 19 member        2023  02               10.5
## 20 member        2023  03               10.2
## 21 member        2023  04               11.5
## 22 member        2023  05               12.6
## 23 member        2023  06               12.9
## 24 member        2023  07               13.2
```

Find mode for ride length for day of the week (rounded in minutes)

```
#define function to calculate mode
find_mode <- function(x) {
  u <- unique(x)
  tab <- tabulate(match(x, u))
  u[tab == max(tab)]
}

#find mode for ride length for day of the week
data_trip %>%
  group_by(member_casual, day_of_week) %>%
  summarize(mode_ride_length = find_mode(round(ride_length)))
```

```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.

## # A tibble: 14 × 3
## # Groups:   member_casual [2]
##    member_casual day_of_week mode_ride_length
##    <chr>         <fct>                  <dbl>
##  1 casual        Sunday                     8
##  2 casual        Monday                     6
##  3 casual        Tuesday                    6
##  4 casual        Wednesday                  6
##  5 casual        Thursday                   6
##  6 casual        Friday                     6
##  7 casual        Saturday                   8
##  8 member        Sunday                     4
##  9 member        Monday                     4
## 10 member        Tuesday                    4
## 11 member        Wednesday                  4
## 12 member        Thursday                   4
## 13 member        Friday                     4
## 14 member        Saturday                   4
```

```
rm(find_mode)
```

Find the coordinates for the top 20 most used start point stations

```
data_trip_start <- data_trip %>%
  group_by(coords_start, start_lat, start_lng) %>%
  summarize(total_count = n()) %>%
  arrange(desc(total_count)) %>%
  as_tibble() %>%
  print(n=20)
```

```
## `summarise()` has grouped output by 'coords_start', 'start_lat'. You can
```

```
## override using the `.groups` argument.

## # A tibble: 2,155,237 × 4
##    coords_start                  start_lat start_lng total_count
##    <chr>                             <dbl>     <dbl>       <int>
##  1 41.892278, -87.612043              41.9     -87.6       45797
##  2 41.880958, -87.616743              41.9     -87.6       27667
##  3 41.911722, -87.626804              41.9     -87.6       24418
##  4 41.89, -87.65                      41.9     -87.6       24242
##  5 41.89, -87.63                      41.9     -87.6       22900
##  6 41.926277, -87.630834              41.9     -87.6       21799
##  7 41.90096039, -87.62377664          41.9     -87.6       21286
##  8 41.902973, -87.63128               41.9     -87.6       21176
##  9 41.94, -87.65                      41.9     -87.6       20579
## 10 41.91, -87.63                      41.9     -87.6       19772
## 11 41.78509714636, -87.6010727606     41.8     -87.6       19380
## 12 41.93, -87.64                      41.9     -87.6       19373
## 13 41.9, -87.63                       41.9     -87.6       19322
## 14 41.791478, -87.599861              41.8     -87.6       19201
## 15 41.912133, -87.634656              41.9     -87.6       18972
## 16 41.88917683258, -87.6385057718     41.9     -87.6       18132
## 17 41.88, -87.63                      41.9     -87.6       17972
## 18 41.93, -87.65                      41.9     -87.6       17373
## 19 41.867888, -87.623041              41.9     -87.6       17249
## 20 41.95, -87.66                      42.0     -87.7       16970
## # i 2,155,217 more rows

data_trip_start <- head(data_trip_start, 20)
```

Find the coordinates for the top 20 most used end point stations

```
data_trip_end <- data_trip %>%
  group_by(coords_end, end_lat, end_lng) %>%
  summarize(total_count = n()) %>%
  arrange(desc(total_count)) %>%
  as_tibble() %>%
  print(n=20)

## `summarise()` has grouped output by 'coords_end', 'end_lat'. You can
override
## using the `.groups` argument.

## # A tibble: 15,314 × 4
##    coords_end                    end_lat end_lng total_count
##    <chr>                           <dbl>   <dbl>       <int>
##  1 41.892278, -87.612043            41.9   -87.6       67800
##  2 41.911722, -87.626804            41.9   -87.6       38663
##  3 41.880958, -87.616743            41.9   -87.6       37933
##  4 41.912133, -87.634656            41.9   -87.6       35926
##  5 41.90096039, -87.62377664        41.9   -87.6       35088
##  6 41.902973, -87.63128             41.9   -87.6       34852
```

```
##  7 41.88917683258, -87.6385057718          41.9   -87.6       33248
##  8 41.926277, -87.630834                   41.9   -87.6       31026
##  9 41.903222, -87.634324                   41.9   -87.6       29912
## 10 41.88338, -87.64117                     41.9   -87.6       29750
## 11 41.9375823160063, -87.6440978050232     41.9   -87.6       29657
## 12 41.8810317, -87.62408432                41.9   -87.6       28222
## 13 41.918306, -87.636282                   41.9   -87.6       27780
## 14 41.791478, -87.599861                   41.8   -87.6       27034
## 15 41.915689, -87.6346                     41.9   -87.6       26056
## 16 41.891466, -87.626761                   41.9   -87.6       25839
## 17 41.9402319181086, -87.6529437303543     41.9   -87.7       25679
## 18 41.929546, -87.643118                   41.9   -87.6       25482
## 19 41.78509714636, -87.6010727606          41.8   -87.6       25284
## 20 41.893992, -87.629318                   41.9   -87.6       24931
## # i 15,294 more rows
```

```
data_trip_end <- head(data_trip_end, 20)
```

---

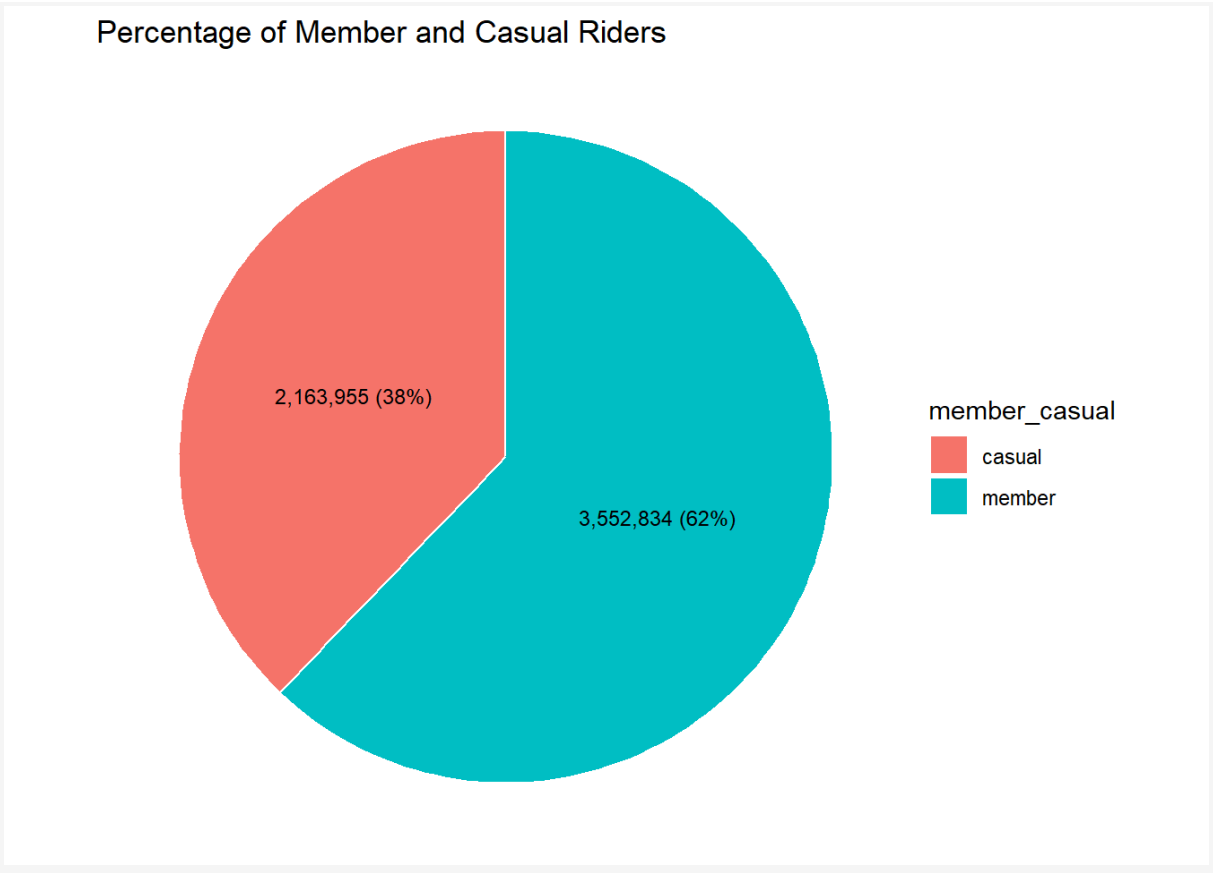
5. Share

Create table with percentages for member and casual riders

```
rider_percentage <- data_trip %>%
  group_by(member_casual) %>%
  summarize(count = n()) %>%
  mutate(percentage = count / sum(count)) %>%
  ungroup()
```

Pie chart of percentage for member and casual rides

```
ggplot(rider_percentage, aes(x="", y=percentage, fill=member_casual)) +
  geom_bar(stat="identity", width=1, color="white") +
  coord_polar("y", start=0) +
  geom_text(aes(label = paste0(comma(count), "
(",scales::percent(percentage),")")), position=position_stack(vjust=0.5),
size = 9/.pt) +
  labs(title="Percentage of Member and Casual Riders") +
  theme_void()
```

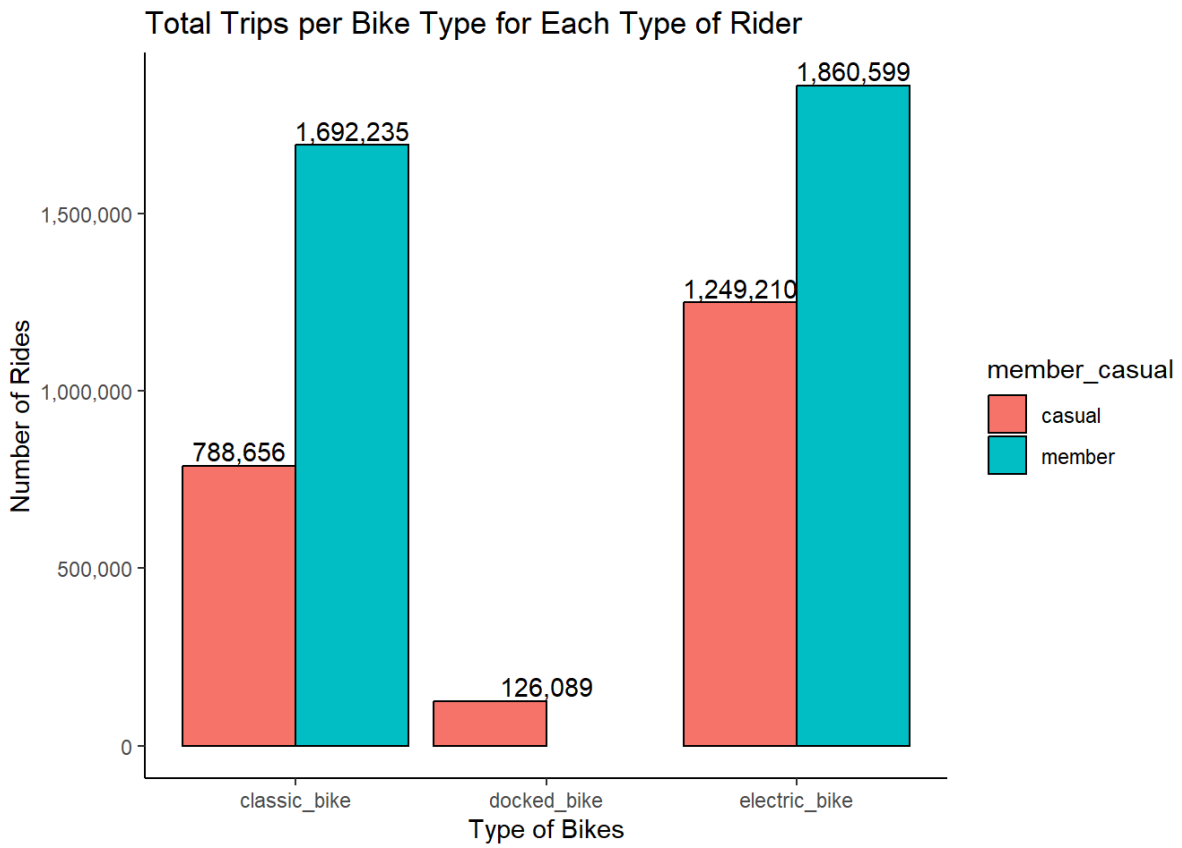



```
rm(rider_percentage)
```

Bar chart for total trips per bike type for each type of rider

```
data_trip %>%
  group_by(member_casual, rideable_type) %>%
  summarize(total_count = n()) %>%
  ggplot(aes(x=rideable_type, y=total_count, fill=member_casual)) +
  geom_bar(position = position_dodge(preserve = "single"), stat="identity",
color="black", width=0.9) +
  scale_y_continuous(labels = scales::comma) +
  geom_text(aes(label=comma(total_count), group=member_casual),
position=position_dodge(width = 0.9), vjust = -0.25) +
  labs(title="Total Trips per Bike Type for Each Type of Rider", x="Type of
Bikes", y="Number of Rides") +
  theme_classic()
```

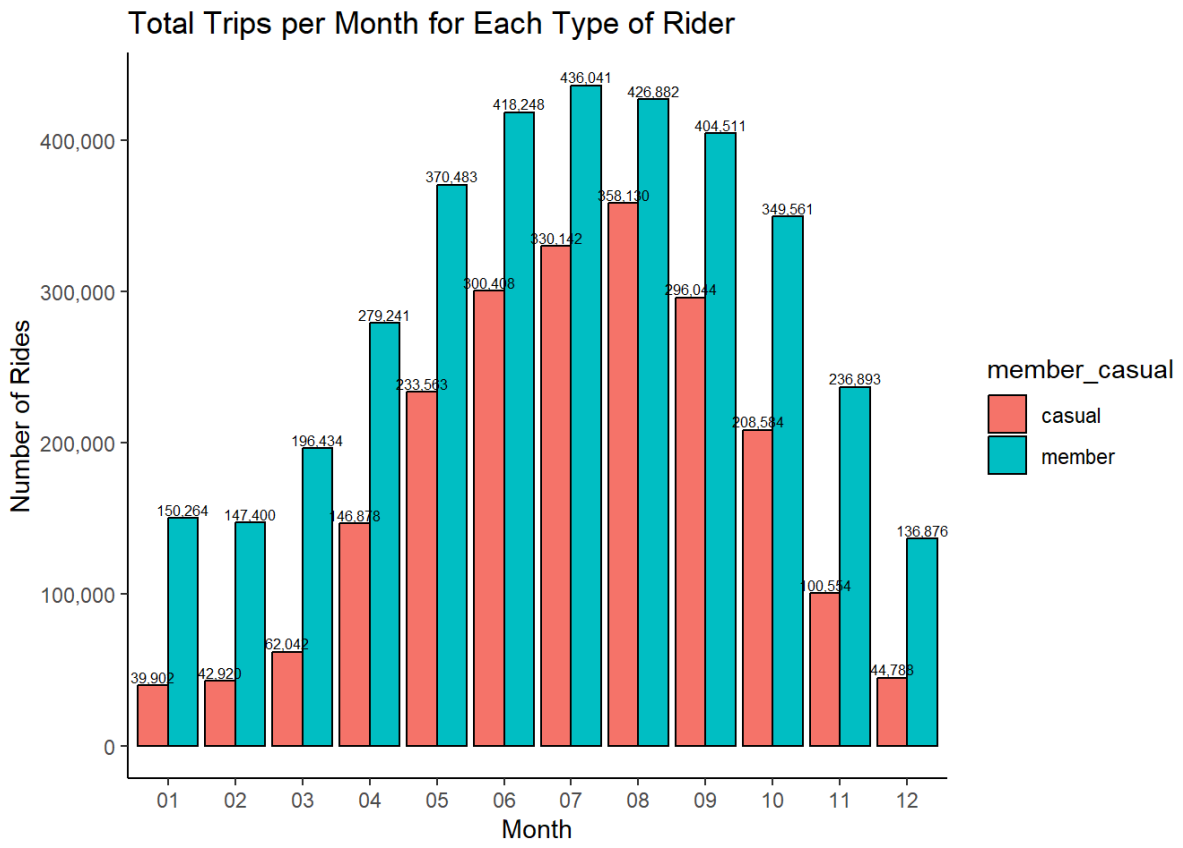
```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```



Bar chart for total trips per month for each rider type

```
data_trip %>%
  group_by(member_casual, month) %>%
  summarize(total_count = n()) %>%
  ggplot(aes(x=month, y=total_count, fill=member_casual)) +
  geom_bar(position = position_dodge(preserve = "single"), stat="identity",
color="black", width=0.9) +
  scale_y_continuous(labels = scales::comma) +
  geom_text(aes(label=comma(total_count), group=member_casual),
position=position_dodge(width = 0.9), vjust = -0.25, size = 6/.pt) +
  labs(title="Total Trips per Month for Each Type of Rider", x="Month",
y="Number of Rides") +
  theme_classic()
```

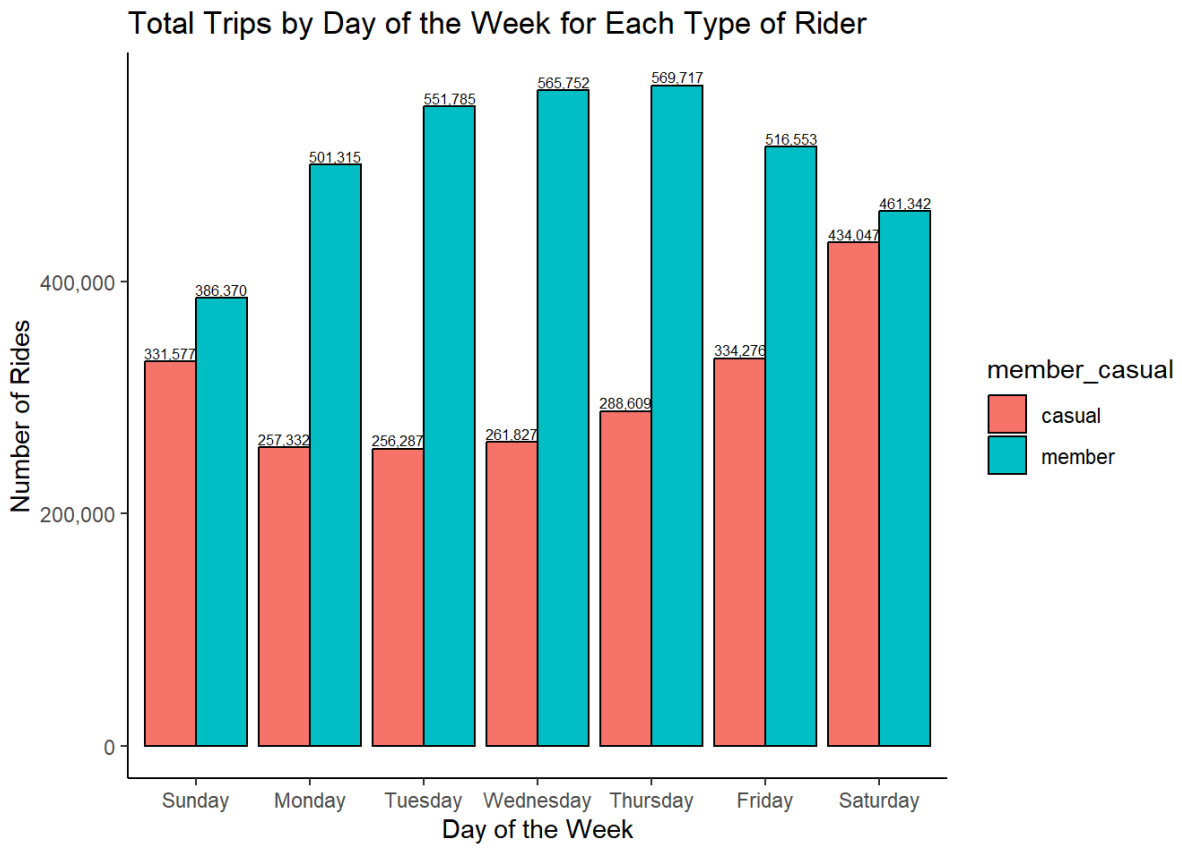
```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```



Bar chart for total trips by day of the week for each rider type

```
data_trip %>%
  group_by(member_casual, day_of_week) %>%
  summarize(total_count = n()) %>%
  ggplot(aes(x=day_of_week, y=total_count, fill=member_casual)) +
  geom_bar(position = position_dodge(preserve = "single"), stat="identity",
color="black", width=0.9) +
  scale_y_continuous(labels = scales::comma) +
  geom_text(aes(label=comma(total_count), group=member_casual),
position=position_dodge(width = 0.9), vjust = -0.25, size = 6/.pt) +
  labs(title="Total Trips by Day of the Week for Each Type of Rider", x="Day
of the Week", y="Number of Rides") +
  theme_classic()
```

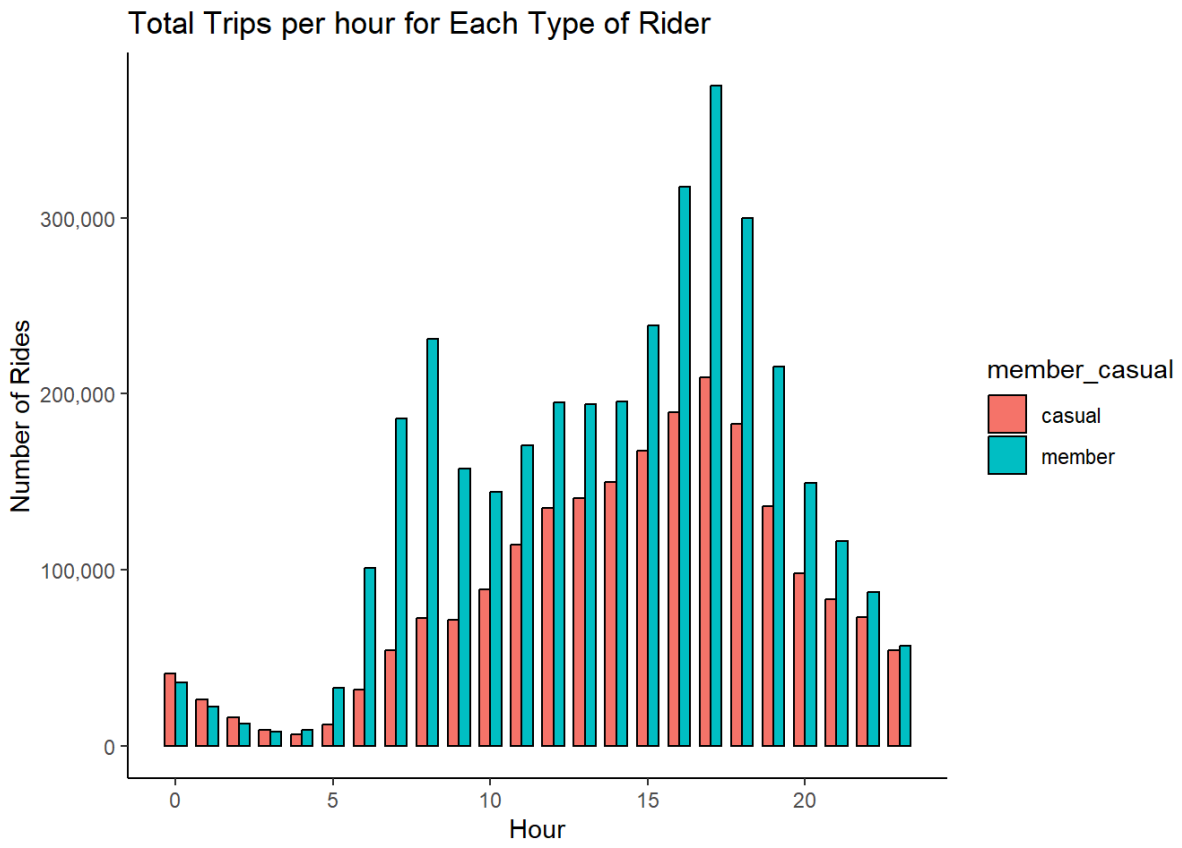
```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```



Bar chart for total trips by hour for each rider type

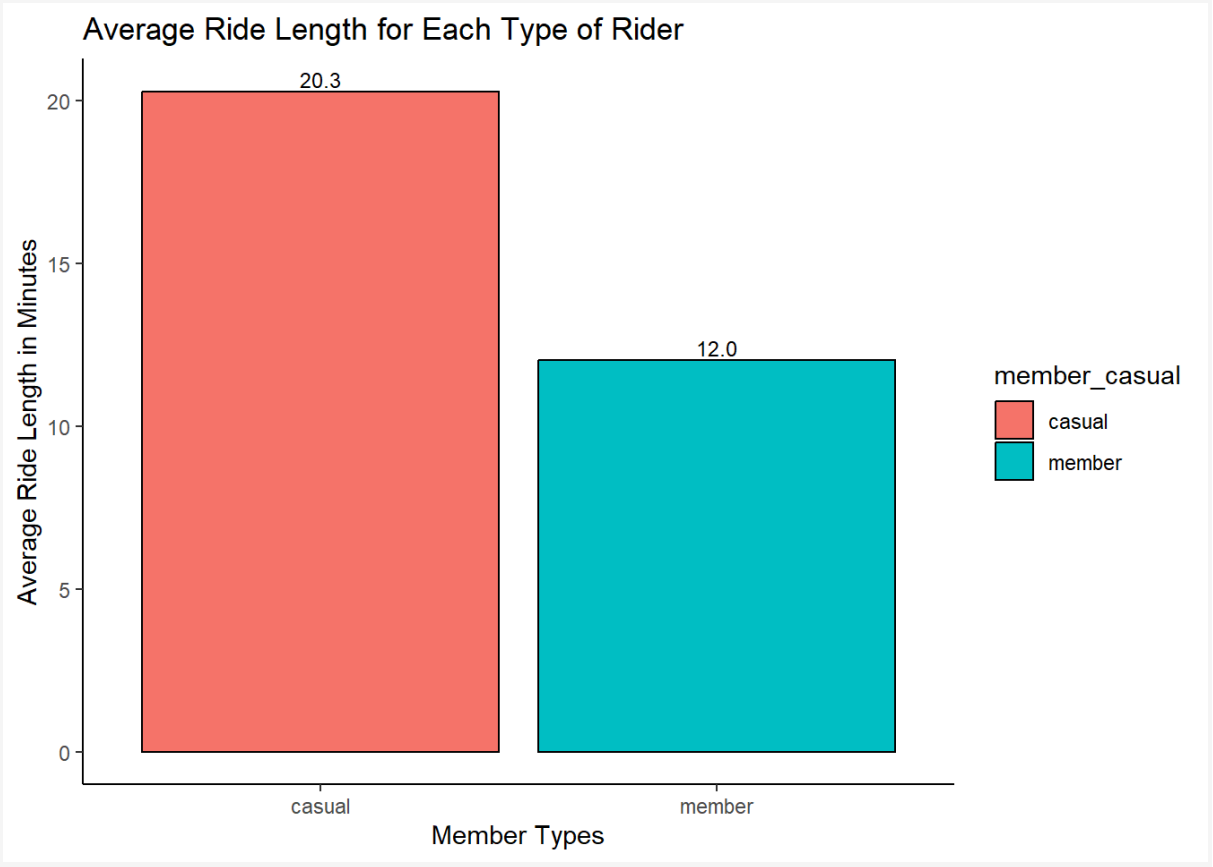
```
data_trip %>%
  group_by(member_casual, hour_started) %>%
  summarize(total_count = n()) %>%
  ggplot(aes(x=hour_started, y=total_count, fill=member_casual)) +
  geom_bar(position = position_dodge(preserve = "single"), stat="identity",
color="black", width=0.7) +
  scale_y_continuous(labels = scales::comma) +
  labs(title="Total Trips per hour for Each Type of Rider", x="Hour",
y="Number of Rides") +
  theme_classic()
```

```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```



Bar chart for average ride length for each rider type (in minutes)

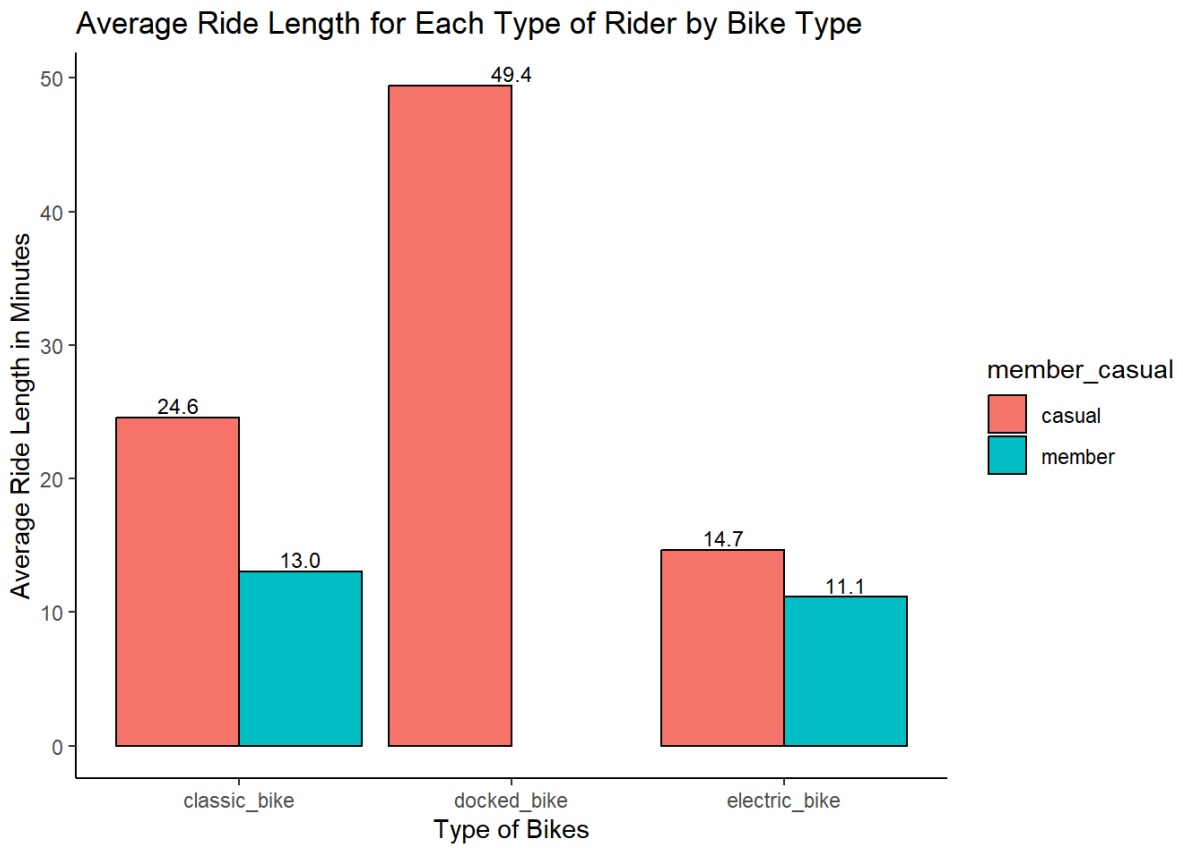
```
data_trip %>%
  group_by(member_casual) %>%
  summarize(avg_ride_length = mean(ride_length)) %>%
  ggplot(aes(x=member_casual, y=avg_ride_length, fill=member_casual)) +
  geom_bar(position = position_dodge(preserve = "single"), stat="identity",
color="black", width=0.9) +
  scale_y_continuous(labels = scales::comma) +
  geom_text(aes(label=comma(avg_ride_length), group=member_casual),
position=position_dodge(width = 0.9), vjust = -0.25, size = 9/.pt) +
  labs(title="Average Ride Length for Each Type of Rider", x="Member Types",
y="Average Ride Length in Minutes") +
  theme_classic()
```



Bar chart for average ride length for each rider type by bike type (in minutes)

```
data_trip %>%
  group_by(member_casual, rideable_type) %>%
  summarize(avg_ride_length = mean(ride_length)) %>%
  ggplot(aes(x=rideable_type, y=avg_ride_length, fill=member_casual)) +
  geom_bar(position = position_dodge(preserve = "single"), stat="identity",
color="black", width=0.9) +
  scale_y_continuous(labels = scales::comma) +
  geom_text(aes(label=comma(avg_ride_length), group=member_casual),
position=position_dodge(width = 0.9), vjust = -0.25, size = 9/.pt) +
  labs(title="Average Ride Length for Each Type of Rider by Bike Type",
x="Type of Bikes", y="Average Ride Length in Minutes") +
  theme_classic()
```

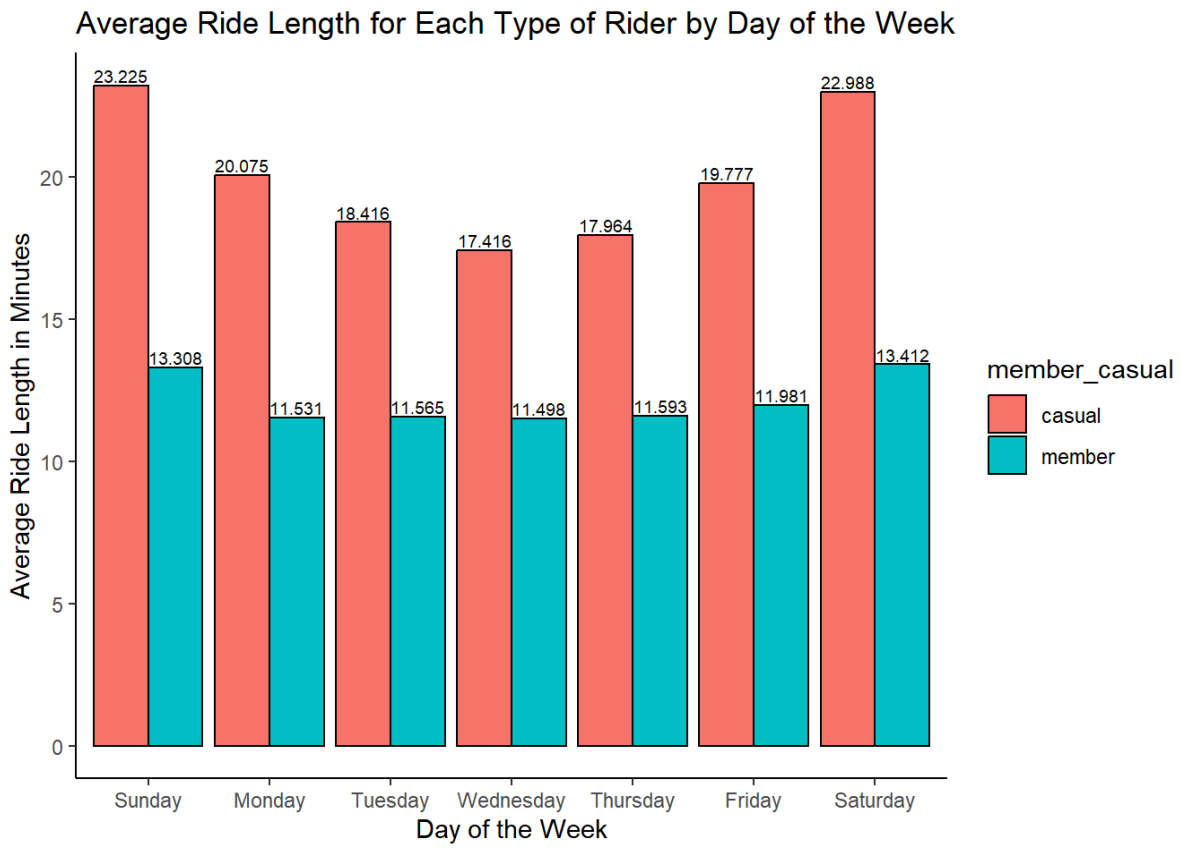
```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```



Bar chart for average ride length for each rider type by day of the week (in minutes)

```
data_trip %>%
  group_by(member_casual, day_of_week) %>%
  summarize(avg_ride_length = mean(ride_length)) %>%
  ggplot(aes(x=day_of_week, y=avg_ride_length, fill=member_casual)) +
  geom_bar(position = position_dodge(preserve = "single"), stat="identity",
color="black", width=0.9) +
  scale_y_continuous(labels = scales::comma) +
  geom_text(aes(label=comma(avg_ride_length), group=member_casual),
position=position_dodge(width = 0.9), vjust = -0.25, size = 7/.pt) +
  labs(title="Average Ride Length for Each Type of Rider by Day of the Week",
x="Day of the Week", y="Average Ride Length in Minutes") +
  theme_classic()
```

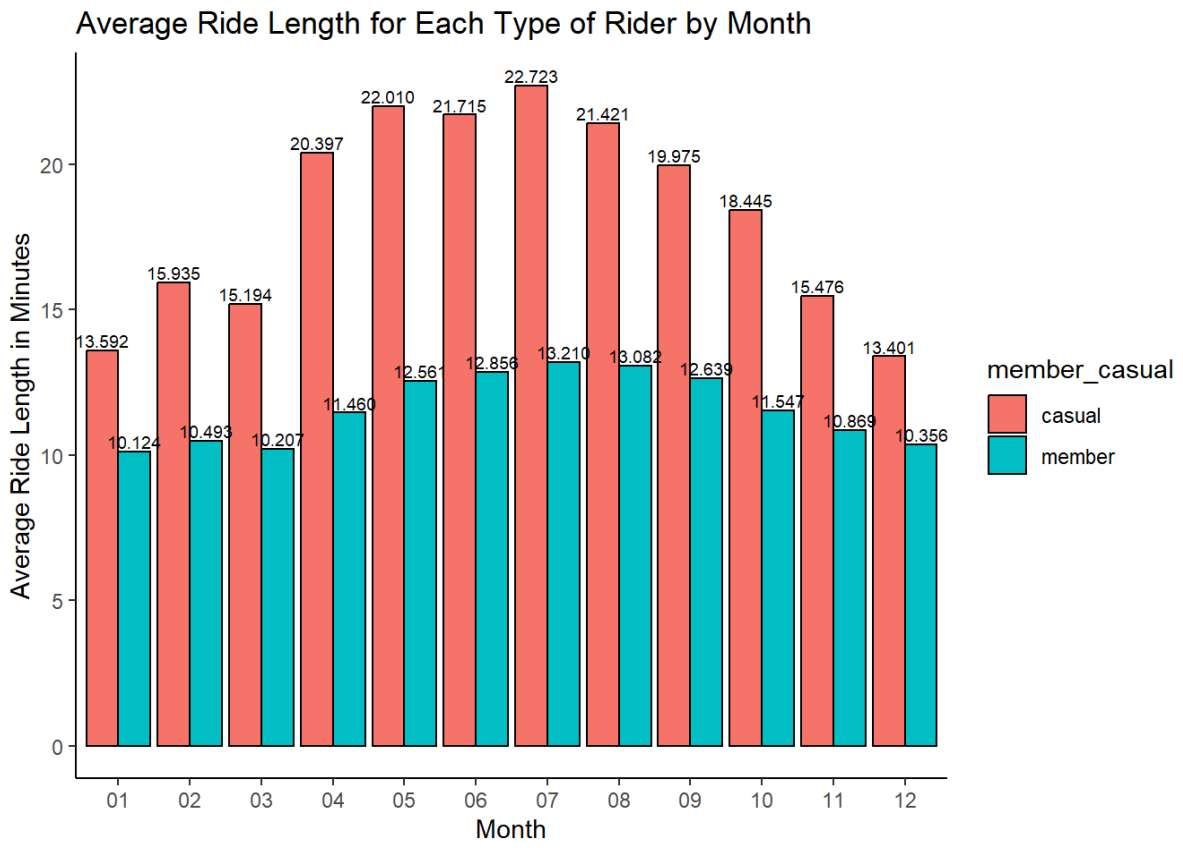
```
## `summarise()` has grouped output by 'member_casual'. You can override
using the
## `.groups` argument.
```



Bar chart for average ride length for each rider type by year and month (in minutes)

```
data_trip %>%
  group_by(member_casual, year, month) %>%
  summarize(avg_ride_length = mean(ride_length)) %>%
  ggplot(aes(x=month, y=avg_ride_length, fill=member_casual)) +
  geom_bar(position = position_dodge(preserve = "single"), stat="identity",
color="black", width=0.9) +
  scale_y_continuous(labels = scales::comma) +
  geom_text(aes(label=comma(avg_ride_length), group=member_casual),
position=position_dodge(width = 0.9), vjust = -0.25, size = 7/.pt) +
  labs(title="Average Ride Length for Each Type of Rider by Month",
x="Month", y="Average Ride Length in Minutes") +
  theme_classic()
```

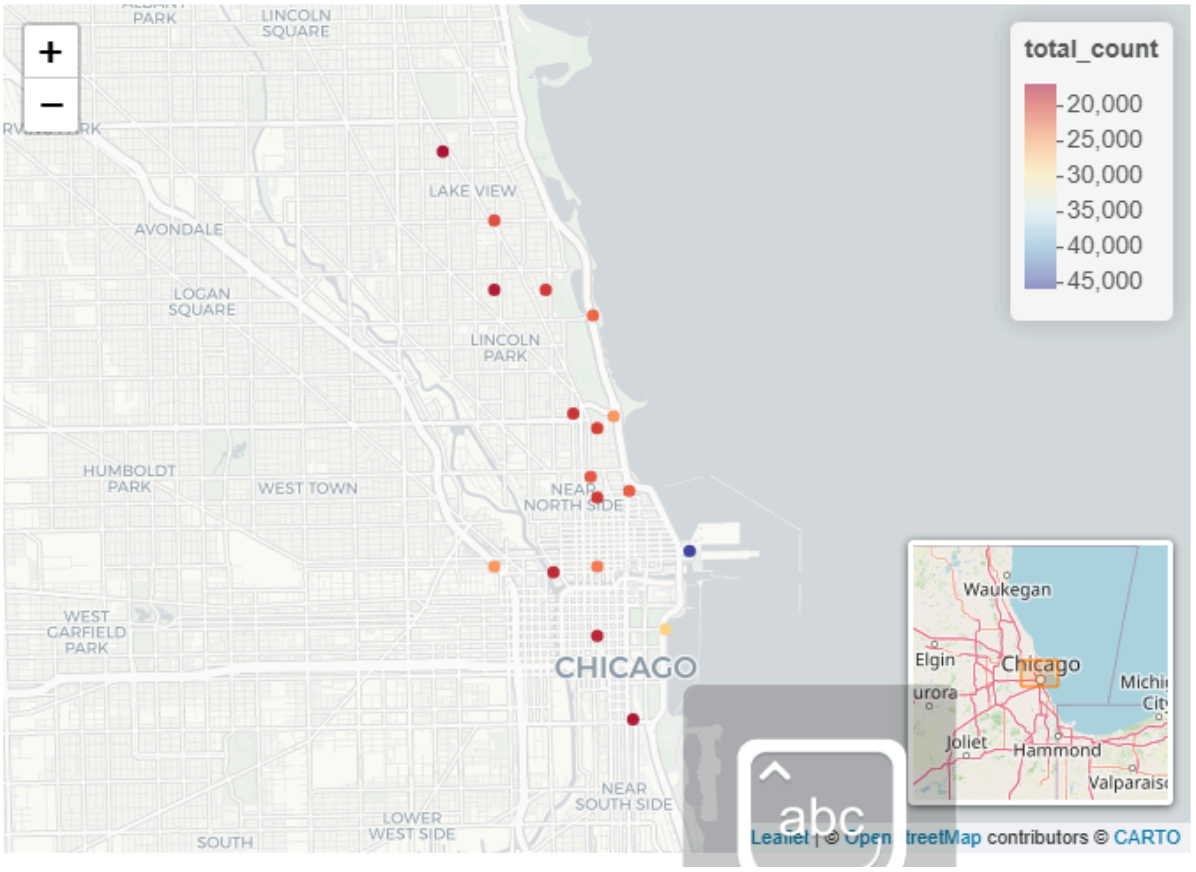
```
## `summarise()` has grouped output by 'member_casual', 'year'. You can
override
## using the `.groups` argument.
```



Heatmap of coordinates for the top 20 most used start point stations

```
pal = colorNumeric("RdYlBu", domain = data_trip_start$total_count)
leaflet(data = data_trip_start) %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircles(lng=~start_lng, lat=~start_lat, opacity = 0.9, color =
~pal(data_trip_start$total_count),

popup=paste(data_trip_start$coords_start,"<br>",comma(data_trip_start$total_c
ount), " start trips")) %>%
  addLegend(pal = pal, values = ~total_count) %>%
  setView(lng = -87.63, 41.91, zoom = 12) %>%
  addMiniMap()
```

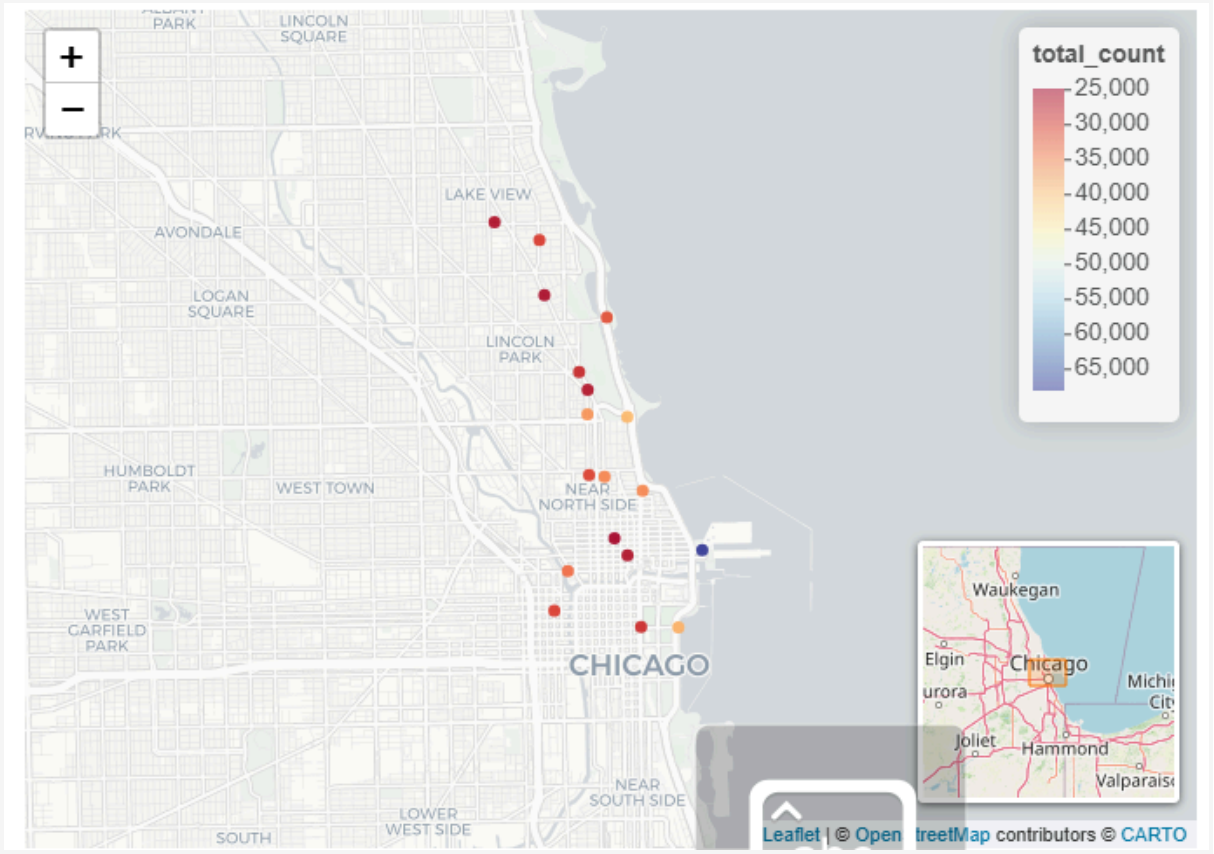



```
rm(data_trip_start)
```

Heatmap of coordinates for the top 20 most used end point stations

```
pal = colorNumeric("RdYlBu", domain = data_trip_end$total_count)
leaflet(data = data_trip_end) %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircles(lng=~end_lng, lat=~end_lat, opacity = 0.9, color =
~pal(data_trip_end$total_count),

popup=paste(data_trip_end$coords_end,"<br>",comma(data_trip_end$total_count),
" end trips")) %>%
  addLegend(pal = pal, values = ~total_count) %>%
  setView(lng = -87.63, 41.91, zoom = 12) %>%
  addMiniMap()
```




```
rm(data_trip_end)
```

```
rm(pal)
rm(data_trip)
ls()
```

```
## character(0)
```

---

6. Act

Conclusion

Ridership was 38% and 62% for casual and member riders, respectively.

Total trips:

- Member riders had more trips during weekdays than weekends
- Casual riders had more trips during weekends than weekdays
- Member and casual riders had more electric bike trips than classic or docked
    - Casual riders on electric bikes nearly doubled classic bikes
    - Member riders took 0 docked bike trips
- Member and casual rider trips were highest during months with warmer weather

Average ride length:

- Casual riders had longer ride length
- Docked bike had longest ride length, followed by classic then electric bikes
    - Member riders ride length were similar
    - Casual riders ride length nearly doubled the next bike type from electric to classic to docked
- Member and casual riders had longer ride length during weekends
    - Casual riders had larger change throughout the week
    - Member riders had consistent ride lengths
- Member and casual riders had longer ride length during warmer months
    - Casual riders had larger change while member riders length were more consistent

Recommendations

Based on analysis and the question of how members and casual riders use Cyclistic bikes differently, the following recommendations will help the marketing team convert casual riders into members:

- June to September have the highest bike usage. Offering summer discounts and incentives for annual membership so trips can be more cost effective.
- A campaign to target weekday casual riders to use bikes to commute to work
- Electric bike usage almost doubled classic bike usage by casual riders. Show the convenience of using electric bikes over other modes of transportation

The coordinates for start and end stations were not used this time per guidelines. The data can be analyzed in the future to develop strategic marketing to target hotspots.